



14.4. Counting Sort

[Previous Section:](#)

 [14.3. Radix Sort](#)

Contents:

[1. The Counting Sort Algorithm](#)

[2. Demonstration of Counting Sort](#)

[3. Implementation of Counting Sort](#)

[Summary](#)

[References](#)

Like **Radix Sort**, **Counting Sort** is also a **non-comparison sorting algorithm**. Instead of repeatedly comparing two elements to determine their order, Counting Sort determines **where each element belongs** in the sorted sequence by counting how many times every value appears.

The main idea is simple:

If we know how many times every value occurs, we can determine the exact position of every element in the final sorted array without performing any comparisons.

Counting Sort is particularly efficient when the values lie within a **small range**, even if the number of elements is relatively large.

1. The Counting Sort Algorithm

Suppose the value **5** appears three times in an array. After sorting, **all three occurrences of 5 must appear consecutively**.

Therefore, instead of comparing every element with every other element, Counting Sort first counts the number of occurrences of every value.

The algorithm consists of four major steps.

- Step 1 – Count the Occurrences: Create an auxiliary array called **count[]**, whose size is one greater than the maximum value.

- Initially:

```
Index : 0 1 2 3 4 5 6 7
Count : 0 0 0 0 0 0 0 0
```

- Each time a value is encountered, increase its corresponding count.
- Step 2 – Compute the Cumulative SumThe count array is then transformed into a **cumulative count array**. Each position stores the number of elements that are **less than or equal** to that value.
 - The cumulative sum is calculated using `count[i] = count[i] + count[i-1]`
 - After this transformation:
 - `count[0]` tells us how many elements are ≤ 0 .
 - `count[1]` tells us how many elements are ≤ 1 .
 - `count[2]` tells us how many elements are ≤ 2 .
 - and so on.
 - This cumulative array allows us to determine the final position of every element.
- Step 3 – Place Each Element: Starting from the **rightmost element** of the original array,
 - Look up its cumulative count.
 - That value indicates the element's final position.
 - Insert the element into the resultant array.

- Decrease its cumulative count by one.

Processing from **right to left** preserves the relative order of duplicate elements, making Counting Sort a **stable sorting algorithm**.

- Step 4 – Obtain the Sorted Array: After every element has been placed into the resultant array, the sorting process is complete.
- Time Complexity: If n = number of elements; k = maximum value in the sequence. Then
 - Time Complexity: $O(n + k)$;
 - Space Complexity: $O(k)$

Counting Sort is therefore very efficient when the range of values is relatively small.

2. Demonstration of Counting Sort

Consider the following sequence.

Input Array

```
+---+---+---+---+---+---+---+---+
| 3 | 2 | 2 | 1 | 7 | 4 | 7 | 1 | 6 | 5 |
+---+---+---+---+---+---+---+---+
```

The values range from **1 to 7**, so we create a count array of size **8 (0-7)**.

- Step 1 – Initialize the Count Array

```
Index : 0 1 2 3 4 5 6 7
Count : 0 0 0 0 0 0 0 0
```

- Step 2 – Count Every Occurrence: Process each element one by one.

```
Count of 3
0 0 0 1 0 0 0 0
```

```
Count of 2
0 0 1 1 0 0 0 0
```

```
Count of 2
0 0 2 1 0 0 0 0
```

```
Count of 1
0 1 2 1 0 0 0 0
```

```
Count of 7
0 1 2 1 0 0 0 1
```

```
Count of 4
0 1 2 1 1 0 0 1
```

```
Count of 7
0 1 2 1 1 0 0 2
```

```
Count of 1
0 2 2 1 1 0 0 2
```

```
Count of 6
0 2 2 1 1 0 1 2
```

```
Count of 5
0 2 2 1 1 1 1 2
```

Final count array

```
Index : 0 1 2 3 4 5 6 7
Count : 0 2 2 1 1 1 1 2
```

Interpretation:

- 1 appears **2** times
- 2 appears **2** times
- 3 appears **1** time
- 4 appears **1** time
- 5 appears **1** time
- 6 appears **1** time
- 7 appears **2** times

- Step 3 – Compute the Cumulative Sum

Now convert the count array into cumulative counts.

Initial array:

```
0 2 2 1 1 1 1 2
```

Update index 1: $2 = 2 + 0$

```
0 2 2 1 1 1 1 2
```

Update index 2: $4 = 2 + 2$

```
0 2 4 1 1 1 1 2
```

Update index 3: $5 = 1 + 4$

```
0 2 4 5 1 1 1 2
```

Update index 4: $6 = 1 + 5$

```
0 2 4 5 6 1 1 2
```

Update index 5: $7 = 1 + 6$

```
0 2 4 5 6 7 1 2
```

Update index 6: $8 = 1 + 7$

```
0 2 4 5 6 7 8 2
```

Update index 7: $10 = 2 + 8$

```
0 2 4 5 6 7 8 10
```

Final cumulative array

```
Index : 0 1 2 3 4 5 6 7
Value : 0 2 4 5 6 7 8 10
```

Interpretation:

- There are **2 elements** ≤ 1
- There are **4 elements** ≤ 2
- There are **5 elements** ≤ 3
- There are **6 elements** ≤ 4
- There are **7 elements** ≤ 5
- There are **8 elements** ≤ 6
- There are **10 elements** ≤ 7

- **Step 4 – Build the Resultant Array**

Create an empty array.

```
Index
```

```
0 1 2 3 4 5 6 7 8 9
```

```
+---+---+---+---+---+---+---+---+---+
|   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+---+
```

We now scan the input array **from right to left**.

Read 5: `count[5] = 7`. Place 5 at index 6

```
+---+---+---+---+---+---+---+---+---+
|   |   |   |   |   | 5|   |   |   |
+---+---+---+---+---+---+---+---+---+
```

count[5] becomes 6

Read 6: `count[6] = 8` . Place at index **7**

```
+---+---+---+---+---+---+---+
|   |   |   |   |   | 5| 6|   |
+---+---+---+---+---+---+---+
```

Read 1: `count[1] = 2` . Place at index **1**

```
+---+---+---+---+---+---+---+
|   | 1|   |   |   | 5| 6|   |
+---+---+---+---+---+---+---+
```

count[1] becomes 1.

Read 7: `count[7] = 10` . Place at index **9**

```
+---+---+---+---+---+---+---+
|   | 1|   |   |   | 5| 6|   | 7|
+---+---+---+---+---+---+---+
```

Read 4: `count[4] = 6` . Place at index **5**

```
+---+---+---+---+---+---+---+
|   | 1|   |   | 4| 5| 6|   | 7|
+---+---+---+---+---+---+---+
```

Read 7: `count[7] = 9` . Place at index **8**

```
+---+---+---+---+---+---+---+
|   | 1|   |   | 4| 5| 6| 7| 7|
+---+---+---+---+---+---+---+
```

```
+---+---+---+---+---+---+---+---+---+---+
```

Read 1: `count[1] = 1` . Place at index **0**

```
+---+---+---+---+---+---+---+---+---+---+
| 1| 1|   |   | 4| 5| 6| 7| 7|
+---+---+---+---+---+---+---+---+---+---+
```

Read 2: `count[2] = 4` . Place at index **3**

```
+---+---+---+---+---+---+---+---+---+---+
| 1| 1|   | 2|   | 4| 5| 6| 7| 7|
+---+---+---+---+---+---+---+---+---+---+
```

Read 2: `count[2] = 3` . Place at index **2**

```
+---+---+---+---+---+---+---+---+---+---+
| 1| 1| 2| 2|   | 4| 5| 6| 7| 7|
+---+---+---+---+---+---+---+---+---+---+
```

Read 3: `count[3] = 5` . Place at index **4**

```
+---+---+---+---+---+---+---+---+---+---+
| 1| 1| 2| 2| 3| 4| 5| 6| 7| 7|
+---+---+---+---+---+---+---+---+---+---+
```

Final Sorted Array

```
+---+---+---+---+---+---+---+---+---+---+
| 1 | 1 | 2 | 2 | 3 | 4 | 5 | 6 | 7 | 7 |
+---+---+---+---+---+---+---+---+---+---+
```

Why Does the Cumulative Array Work?

Consider the cumulative value of **7**, which is **10**. This means there are **10 elements less than or equal to 7**. Therefore, the **last occurrence** of 7 must be placed at index `10 - 1 = 9`

Similarly,

- `cumulative(6) = 8` → last 6 is placed at index 7.
- `cumulative(5) = 7` → last 5 is placed at index 6.
- `cumulative(2) = 4` → last 2 is placed at index 3.

Whenever an element is placed, its cumulative count decreases by one, allowing duplicate elements to occupy consecutive positions while preserving their original order.

3. Implementation of Counting Sort

Below is the implementation of Counting Sort using Python

```
def counting_sort(a): # [Time(n), Space(n) = O(n + k), O(k)]
    # Create a count array
    max_array = max(a)
    count_array = [0] * max_array
    # Count the occurrence of each element in array
    for i in a:
        # Store them in the count array based on index
        count_array[i - 1] += 1
    # Iteratively compute the elements closest to each other in array
    for j in range(1, len(count_array)):
        count_array[j] += count_array[j - 1]
    # Construct the sort array
    sort_array = [0] * len(a)
    for k in range(len(a) - 1, -1, -1):
        # Shift the computed occurrence
        sort_array[count_array[a[k] - 1] - 1] = a[k]
```

```
count_array[a[k] - 1] -= 1

return sort_array
```



Counting Sort in Radix Sort

Although Counting Sort is a complete sorting algorithm on its own, it is also commonly used as the underlying sorting method in **Radix Sort**. Instead of creating ten physical buckets (0–9), each pass of Radix Sort can use Counting Sort to arrange the elements according to the current digit (units, tens, hundreds, etc.).

This combination is highly efficient because each digit can only have a value from **0 to 9**, so the counting array always contains **10 elements**, regardless of how large the numbers themselves are. As a result, each digit can be sorted in linear time while maintaining the stability required by Radix Sort, making **Radix Sort with Counting Sort** one of the most efficient algorithms for sorting large collections of integers with a fixed number of digits.

Summary

Unlike comparison-based sorting algorithms, Counting Sort determines the final position of each element by counting how many times every value appears.

- It first constructs a frequency array, transforms it into a cumulative count array, and then places every element directly into its correct position in the resultant array.
- Because the original array is processed from **right to left**, duplicate elements preserve their relative order, making Counting Sort a **stable sorting algorithm**.

Its time complexity is $O(n + k)$, making it highly efficient when the range of input values is relatively small.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqqBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/counting-sort/>

<https://www.programiz.com/dsa/counting-sort>

<https://www.youtube.com/watch?v=OKd534EWcdk>

Next Section:

 [14.5. Merge Sort](#)